

Cross-Rover Semantic Segmentation of Mars NavCam Terrain with Domain Adaptation

Cesare Fraccaroli

Computer Vision and Deep Learning
Università degli Studi di Verona (UniVR)

AI4MARS /MSL→MER /ResNet34-U-Net /July 2026

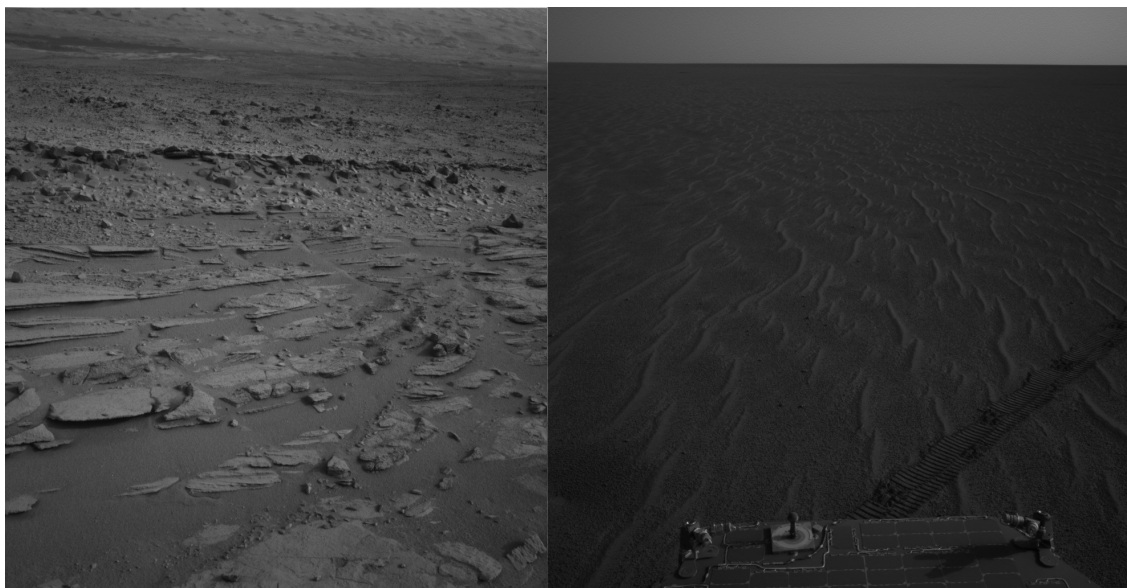
Abstract

We study semantic segmentation of Mars navigation camera images into four terrain classes (*soil*, *bedrock*, *sand*, *big_rock*) on **AI4MARS**. A **ResNet34 U-Net** is pretrained on **MSL** (Curiosity) and adapted to **MER** (Spirit/Opportunity) with feature-level **UDA**, **pseudo-labeling** and **Mean Teacher** training. **Pseudo-labeling** reaches **0.869 wIoU** and Mean Teacher reaches 0.769 wIoU. The rare *big_rock* class remains the weakest among the experiments.

Motivation and Rationale

Autonomous Mars rovers such as Curiosity (noted as MSL) and Spirit/Opportunity (noted as MER) rely on navigation cameras for hazard detection, path planning and terrain segmentation. Pixel-wise terrain segmentation maps each image to different kind of terrains so the rover can trace the optimal path.

The public *AI4MARS* dataset [1] provides expert labels across missions, but **domain shift** is present between them: camera geometry, lighting, terraing type differ between rovers. A model trained only on MSL (Curiosity) does not perform very well on MER (but not exactly bad). We want to improve the task performance.



(a) MSL

(b) MER

Figure 1: MSL vs MER appearance.

Why this matters

As for all segmentation tasks, manual labeling of Mars images is expensive and MER has far fewer labeled NavCam frames than MSL. **Unsupervised domain adaptation (UDA)** leverages large unlabeled MER pools (**2000** images in our protocol) to try to adapt the different domains. We then try to introduce a small labelled MER dataset in order to leverage semi supervised methods such as **pseudo-labeling** and **Mean Teacher** to see if, and how much, the task can be improved. We used only a subset of AI4MARS for computational reasons.

Rationale for our pipeline design

Instead of training a U-Net from scratch we adopted an architecture using a **ResNet34** encoder inside the U-Net architecture. With this approach we can focus on fine tuning on the specific dataset without the expensive need of training on big dataset since ResNet34 is already pretrained on **ImageNet**.

Problem

Unsupervised domain adaptation (UDA) on MER: source domain $\mathcal{D}_S$ (**MSL**, labeled), target domain $\mathcal{D}_T$ (**MER**, mostly unlabeled), shared label space $\{0, 1, 2, 3\}$ plus ignore mask. The goal is to learn a segmenter f_θ that minimizes **target risk** without access to MER test labels at training time, while preserving source performance as a sanity check.

Table 1: Labeled pixel share by class (%).

Subset	soil	bedrock	sand	big_rock
MSL (<code>msl_train</code>)	38.2	49.7	11.2	0.9
MER (<code>mer_test</code>)	41.8	27.4	29.8	1.1

One of the important issues in the task is the imbalance of the classes. The **big_rock** class is about **1%** of labeled pixels on both rovers, while soil, bedrock and sand cover almost all of the rest. That imbalance makes **big_rock** IoU noisy and hard to learn: a few wrong pixels move the score a lot, and UDA alone barely helps until a small **labeled MER** set is introduced.

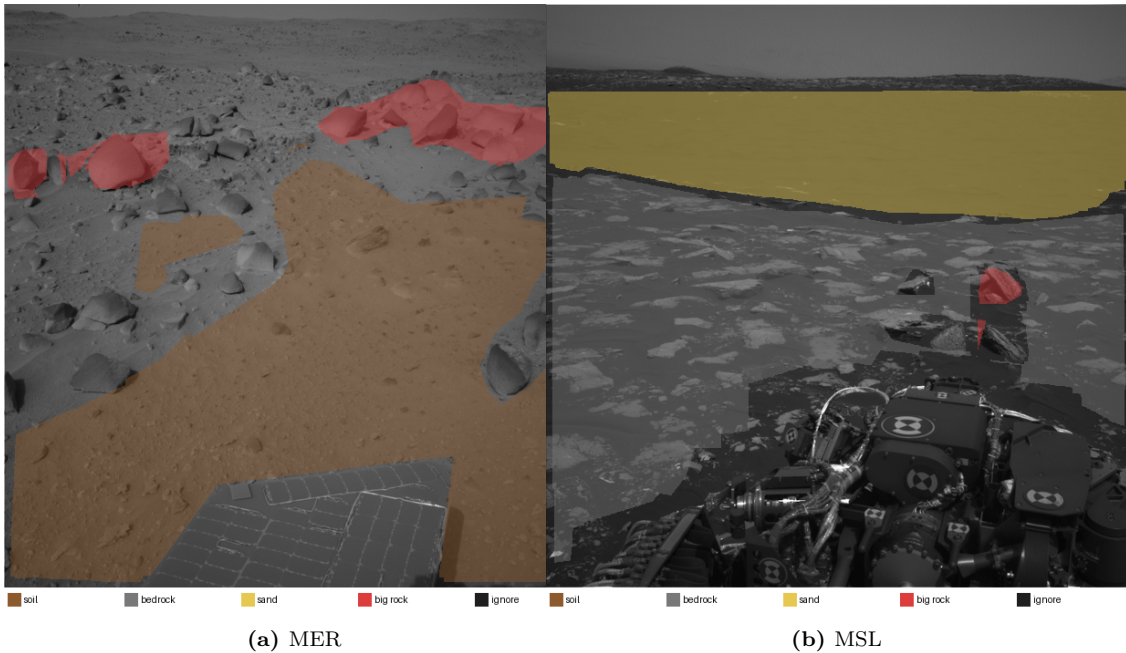


Figure 2: MER labels (image and mask).

State of the Art

Semantic segmentation

Fully convolutional networks and U-Net [7] established encoder-decoder architectures with skip connections for dense prediction. ResNet backbones [5] with U-Net decoders are a standard transfer-learning recipe on remote-sensing and robotics imagery: ImageNet pretraining supplies low-level and precise filters. The decoder learns task-specific boundaries.

Vision Transformers and foundation models

Vision Transformers (ViT)

ViT [4] approaches an image as a sequence of fixed-size patches which are processed by transformer encoder blocks with global self-attention. Variants like hybrid CNN-transformer decoders (e.g. SegFormer) try to mix long-range context with multi-scale feature maps. However, transformers generally need large pretraining datasets and use more GPU memory than compact CNNs, especially when working at 512-pixel resolution. For our project this made ViT a less practical choice not only would we need heavier computational resources, but adapting all of our UDA experiments to a new architecture would have increased much the magnitude of the project.

Segment Anything Model (SAM)

SAM [12] is a promptable segmentation foundation model developed by Meta, trained on over one billion masks. Given point, box or coarse prompts it produces high quality object masks with strong zero-shot generalization on natural images. Follow ups (SAM 2, domain-specific fine-tunes) extend this idea to video and specialized sensors.

Relevance to Mars terrain segmentation

While SAM and ViT-based pipelines represent some of the current state-of-the-art architectures for segmentation, our project faces some important differences and constraints. In our setting:

1. AI4MARS defines four terrain classes rather than prompt-based instance masks as in SAM.
2. Adaptation from labeled MSL to mostly unlabeled MER is the **main difficulty**.
3. Training budget and labeled MER data are limited, so a **ResNet34-U-Net** is more practical than a large transformer or SAM.
4. CORAL, MMD and DANN fit CNN encoders by aligning the U-Net bottleneck (**enc4**); porting the same UDA stack to SAM or ViT would need extra work.
5. Foundation models could still serve as frozen encoders or pseudo-label teachers on unlabeled MER frames.

Unsupervised domain adaptation SOTA

Vision-language models and CLIP

CLIP [6] pretrains joint image-text embeddings from web-scale captions, so new categories can be queried at inference with text prompts (e.g. class names) instead of extra labeled data. Newer VLMs (ALIGN, SigLIP, multimodal LLMs) add stronger language conditioning. In segmentation and UDA, these models can provide semantic targets for mask generation, pseudo-labels on unlabeled target images or text-based alignment when both domains share the same class names.

Foundation-model and prompt-based UDA trends

A common trend in domain adaptation (UDA) is to use big pretrained models like CLIP or ViT as fixed backbones, and just adjust (or finetune) small extra layers or tokens for the new domain. When there isn't much labeled data, researchers often let these models generate their own labels to help with training. There's also growing interest in using models like diffusion or SAM to generate rough masks on new data and then train smaller segmentation models with those masks and a handful of real labels.

Our choices for UDA

We adapt MSL→MER with **feature-level** methods on shared **enc4** representations. The methods we decided to implement are:

- **CORAL** [8]: covariance alignment on dense **enc4** feature maps.
- **MMD** [3]: RBF-kernel MMD on dense **enc4** features.
- **DANN** [2, 14]: domain discriminator on GAP(**enc4**) with GRL, joint with MSL segmentation loss and unlabeled MER batches.
- **ADDA** [9, 15]: two-phase adversarial adaptation: train a domain discriminator on frozen GAP features (phase 1), then adapt a *shared* encoder with MSL segmentation, MER adversarial alignment and dense CORAL on **enc4** (phase 2).

CORAL, MMD and DANN use $\mathcal{L} = \mathcal{L}_{\text{seg}} + \lambda \mathcal{L}_{\text{DA}}$ ($\lambda=0.1$) from the pretraining checkpoint. ADDA uses a dedicated 15-epoch schedule (5 discriminator + 10 adaptation) with $\lambda_{\text{adv}}=0.1$ (Ganin ramp) and $\lambda_{\text{CORAL}}=0.1$. Our DANN and ADDA loops follow public PyTorch references [14, 15], adapted from classification to ResNet34-U-Net segmentation (shared encoder for ADDA). All checkpoints are selected on **mer_val** at 512-pixel NavCam resolution.

DANN vs ADDA

DANN updates encoder and discriminator jointly in one step via gradient reversal. ADDA pre-trains the discriminator, then alternates D and encoder updates in phase 2. We use a shared encoder (not a separate target copy) and add dense CORAL, which avoids the MER val collapse seen with a textbook separate-encoder ADDA setup on segmentation.

Pseudo-labeling and Mean Teacher segmentation

Pseudo-labeling [13, 16] assigns hard labels to confident unlabeled pixels and combines their loss with supervised MER cross-entropy. **Mean Teacher** [10, 17] instead matches the student's predictions to an exponential-moving-average teacher predictions. Both methods use the same **200 labeled MER** images as a supervised base. Our code follows the public PyTorch references above, adapted to dense prediction (confidence masking for pseudo-labels; MSE on softmax maps for Mean Teacher).

Objectives

Primary objective

Train a segmenter on MSL and adapt it to MER such that **MER test wIoU** exceeds an MSL-pretrained baseline while keeping MSL test performance reasonable.

Secondary objectives

1. Compare feature-level UDA (CORAL, MMD, DANN, ADDA) and pseudo-labeling and Mean Teacher under a common encoder-decoder architecture.

2. Analyze class behaviour, especially the rare *big_rock* class.

Methodology

Dataset and preprocessing

We use the *AI4MARS* corpus [1]: RGB NavCam images with four terrain classes (*soil*, *bedrock*, *sand*, *big_rock*). Rover body and uncertain pixels map to `ignore_index= 255` and are excluded from loss and metrics. Images are resized to 512×512 with and ImageNet-normalized. Labels use **nearest-neighbour** resizing so class indices stay discrete (basically we round the values to the nearest integer). On MSL only, auxiliary `mxy` (rover body) and `rng-30m` (beyond about 30 m) masks set additional pixels to ignore, avoiding training on the rover or distant regions where manual annotation was deliberately omitted in AI4MARS.

Table 2: Data splits.

Split	Role	N
<code>msl_train / val / test</code>	Pretrain / test	3000 / 322
<code>mer_labeled_adapt</code>	Supervised MER anchor	200
<code>mer_uda</code>	Unlabeled MER (UDA + pseudo)	2000
<code>mer_val / mer_test</code>	Select adaptation / final test	200 / 204

Validation protocol

1. Tune on the appropriate validation split.
2. Early-stop on validation mIoU to avoid overfitting.
3. Report test metrics *once* for selected finalists only.

Pretrain hyperparameter search uses `msl_val`.

Table 3: Pretraining hyperparameters.

Set	Encoder LR	Decoder LR	Weight decay
Base	10^{-5}	10^{-4}	10^{-4}
Low LR (selected)	5×10^{-6}	5×10^{-5}	10^{-4}
Higher decoder LR	10^{-5}	2×10^{-4}	10^{-4}

Metrics

Our primary reported score is frequency-weighted IoU (**wIoU**). With per-class intersection-over-union IoU_c and ground-truth pixel counts n_c on the evaluated split,

$$\text{wIoU} = \sum_{c=1}^C f_c \text{IoU}_c, \quad f_c = \frac{n_c}{\sum_{c'=1}^C n_{c'}}. \quad (1)$$

wIoU weights each class by how often it appears in the image, so it tracks performance on the pixels that dominate the scene. Unweighted mean IoU (**mIoU**), $\text{mIoU} = \frac{1}{C} \sum_c \text{IoU}_c$, and per-class IoU are kept in tables and plots for completeness: equal class weights can overstate rare-class failures such as *big_rock* (about 1% of MER test pixels) relative to the frequent terrain classes.

Architecture

We couple an ImageNet-pretrained ResNet34 encoder [5] with a four-block U-Net decoder [7] that possesses a segmentation head. Skip connections link encoder stages to matching decoder levels. The bottleneck **enc4** has shape $512 \times 16 \times 16$ for a 512×512 input. `encode()` / `decode()` methods in the codebase split the forward path so UDA can target **enc4** without breaking segmentation. **Feature-level UDA** is applied at this bottleneck. Pretraining uses `lr_encoder`= 10^{-5} and `lr_decoder`= 10^{-4} (AdamW).

Encoder-decoder data flow

Input RGB passes through ResNet stem and layers 1-4, producing skip tensors. The decoder upsamples from **enc4**, concatenating skips at each level and applying two 3×3 conv blocks per stage. A final upsample restores 512×512 resolution before the classification head. Only **enc4** is domain-aligned. Skip tensors retain source-domain fine structure, which helps MSL accuracy but limits how much MER appearance can change in shallow layers.

Loss functions and training stages

All training stages minimize cross-entropy on valid pixels (`ignore_index=255` excluded). Let f_θ denote the segmenter, $\Omega(Y)$ the set of labeled pixels in mask Y , and $\text{CE}(\hat{y}, y)$ the per-pixel cross-entropy. For a batch

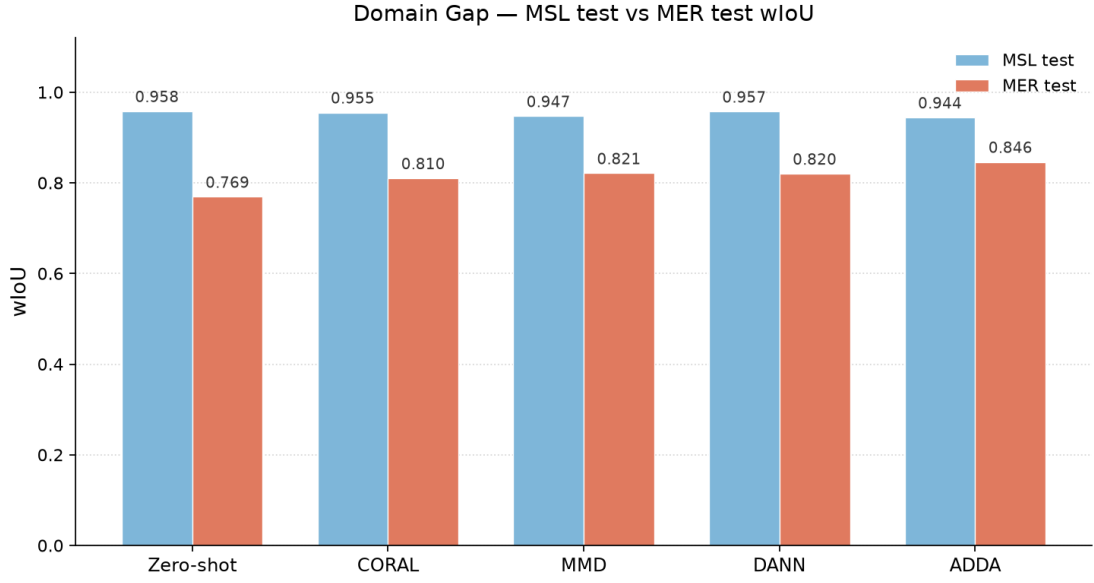


Figure 3: Domain gap on mIoU (MSL test vs MER test).

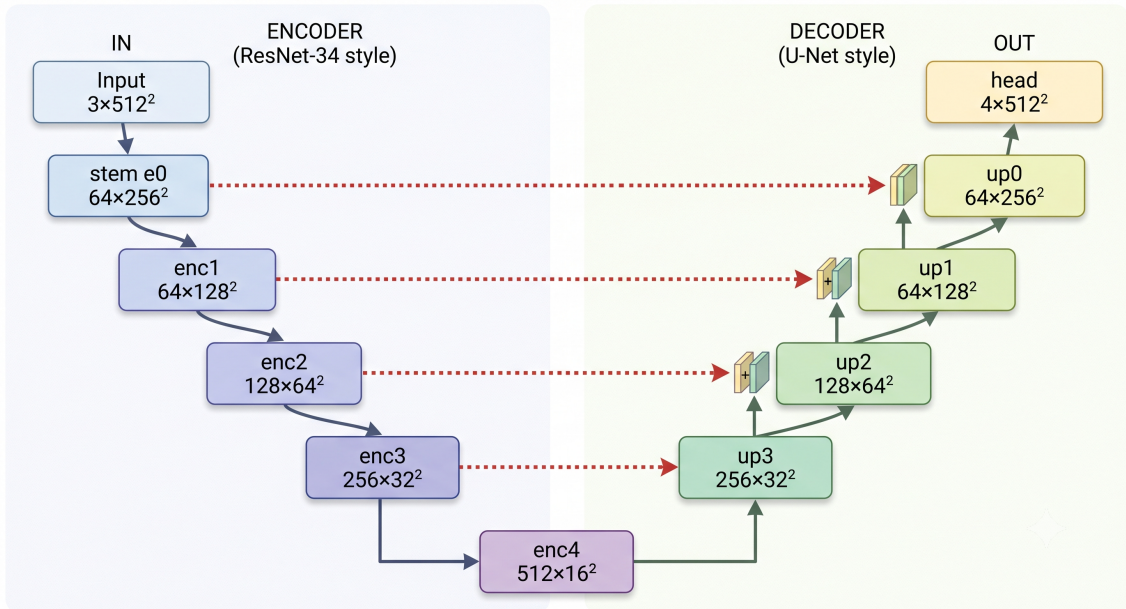


Figure 4: ResNet34-U-Net with feature UDA at enc4.

of images X and labels Y ,

$$\mathcal{L}_{\text{seg}}(X, Y) = \frac{1}{|\Omega(Y)|} \sum_{p \in \Omega(Y)} \text{CE}(f_{\theta}(X)_p, y_p). \quad (2)$$

UDA methods extract bottleneck tensors $\mathbf{z} = \text{enc4}(X) \in \mathbb{R}^{512 \times 16 \times 16}$, flatten them to per-location features $\phi(\mathbf{z}) \in \mathbb{R}^{N \times C}$ ($N=256$, $C=512$) with L2 normalization along C and apply domain losses on ϕ or on global-average-pooled vectors $h(\mathbf{z}) = \text{GAP}(\mathbf{z}) \in \mathbb{R}^{512}$.

Segmentation and pretraining

MSL pretraining minimizes $\mathcal{L}_{\text{seg}}$ on `msl_train` only. The best checkpoint on `msl_val` initializes all adaptation runs. Hyperparameter search (Table 3) compares learning rates and weight decay;

Feature-level UDA

From the pretraining checkpoint, each iteration pairs labeled MSL batches (X_S, Y_S) with unlabeled MER images X_T and optimizes

$$\mathcal{L}_{\text{UDA}} = \mathcal{L}_{\text{seg}}(X_S, Y_S) + \lambda \mathcal{L}_{\text{DA}}, \quad \lambda = 0.1, \quad (3)$$

with $\mathcal{L}_{\text{DA}}$ chosen as follows (all on `enc4 feat_map` tensors unless noted).

CORAL [8] aligns second-order statistics of source and target features. With centered covariance matrices C_S, C_T of $\phi(\mathbf{z}_S)$ and $\phi(\mathbf{z}_T)$,

$$\mathcal{L}_{\text{CORAL}} = \frac{1}{4C} \|C_S - C_T\|_F^2. \quad (4)$$

MMD [3] uses an unbiased RBF-kernel estimate ($\sigma=1.0$, subsample cap 2048):

$$\mathcal{L}_{\text{MMD}} = \mathbb{E}[k(\phi_s, \phi'_s)] + \mathbb{E}[k(\phi_t, \phi'_t)] - 2\mathbb{E}[k(\phi_s, \phi_t)], \quad k(u, v) = \exp\left(-\frac{\|u-v\|^2}{2\sigma^2}\right). \quad (5)$$

DANN [2] attaches a domain discriminator D to $h(\mathbf{z})$ with gradient reversal (GRL) strength λ_p ramped by the Ganin schedule ($\lambda_{\text{max}}=0.1$):

$$\mathcal{L}_{\text{DANN}} = \mathcal{L}_{\text{seg}}(X_S, Y_S) + \text{BCE}(D(\text{GRL}_{\lambda_p}(h_S)), \mathbf{0}) + \text{BCE}(D(\text{GRL}_{\lambda_p}(h_T)), \mathbf{1}), \quad (6)$$

where source labels are 0 and target labels are 1. D and the encoder are updated jointly each step.

Checkpoints for CORAL, MMD and DANN are selected on `mer_val`.

ADDA adaptation

ADDA adaptation [9] runs in two phases from the same pretrain checkpoint.

Phase 1 (discriminator pre-training). The segmenter is frozen. D is trained on $\text{GAP}(\text{enc4})$ features from paired MSL/MER batches:

$$\mathcal{L}_D = \text{BCE}(D(h_S), \mathbf{0}) + \text{BCE}(D(h_T), \mathbf{1}). \quad (7)$$

Phase 2 (encoder adaptation). Each iteration alternates an update of D on detached features (Eq. 7) with an encoder/decoder step that minimizes

$$\mathcal{L}_{\text{ADDA}} = \lambda_{\text{seg}} \mathcal{L}_{\text{seg}}(X_S, Y_S) + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} + \lambda_{\text{CORAL}} \mathcal{L}_{\text{CORAL}}, \quad (8)$$

where $\mathcal{L}_{\text{adv}} = \text{BCE}(D(h_T), \mathbf{0})$ pushes MER features toward the source domain, $\mathcal{L}_{\text{CORAL}}$ is Eq. 4 on dense `enc4` maps, $\lambda_{\text{seg}}=1$, λ_{adv} is ramped (Ganin schedule, peak 0.1) and $\lambda_{\text{CORAL}}=0.1$. The selected recipe uses 5 discriminator epochs, 10 adaptation epochs and batch size 32.

Pseudo-labeling

On unlabeled MER images X_U , the student produces per-pixel labels $\hat{y}_p = \arg \max_c f_{\theta}(X_U)_{p,c}$. Pixels with maximum softmax confidence below a certain threshold (we use $\tau=0.9$) are ignored. Each update pairs one labeled MER batch with one unlabeled batch and minimizes

$$\mathcal{L}_{\text{PL}} = \mathcal{L}_{\text{seg}}(X_L, Y_L) + \alpha(t) \mathcal{L}_{\text{seg}}(X_U, \hat{Y}_U), \quad (9)$$

where $\alpha(t)$ increases from 0.25 to 1 over the first four epochs. Pseudo-labels come from the current student.

Mean Teacher

Mean Teacher keeps a teacher network with parameters $\theta' \leftarrow 0.99 \theta' + 0.01 \theta$. The student receives supervised cross-entropy on the **200 labeled MER** images. On unlabeled MER, student and teacher are encouraged to produce the same predictions:

$$\mathcal{L}_{\text{MT}} = \mathcal{L}_{\text{seg}}(X_L, Y_L) + \lambda(t) \|\text{softmax}(f_{\theta}(X_U + \eta_s)) - \text{softmax}(f_{\theta'}(X_U + \eta_t))\|_2^2. \quad (10)$$

The consistency weight follows the standard sigmoid ramp $\lambda(t) = \exp[-5(1-t)^2]$ during the first five epochs.

Experiments and Results

Experimental setup

All runs use batch size 16 (around 12GB of peak GPU memory usage) and AdamW optimizer. Full training used an NVIDIA RTX 4090 (24 GB) hosted on the cloud with Runpod service for speed.

Implementation overview

All experiments share one PyTorch pipeline built around the same ResNet34-U-Net segmenter and every method reuses this as baseline and differs only in the training objective (MSL pretraining, feature-level alignment on `enc4`). The two semi-supervised methods use different data splits for evident reasons.

Pretraining

MSL pretraining on `msl_train` reaches 0.753 validation mIoU. The selected checkpoint is evaluated as a baseline on MER before any adaptation (Table 4, baseline row).

Main pipeline

Table 4 summarizes test scores after one validation-selected evaluation per method. Baseline MER test wIoU is **0.769**. Among UDA methods, **ADDA** reaches the highest MER wIoU (**0.846**), ahead of MMD (0.821), DANN (0.820) and CORAL (0.810).

Table 4: Test scores on MSL and MER.

Method	MSL test		MER test	
	mIoU	wIoU	mIoU	wIoU
Baseline	0.801	0.958	0.590	0.769
CORAL	0.797	0.955	0.623	0.810
MMD	0.772	0.947	0.639	0.821
DANN	0.828	0.957	0.637	0.820
ADDA	0.857	0.944	0.647	0.846

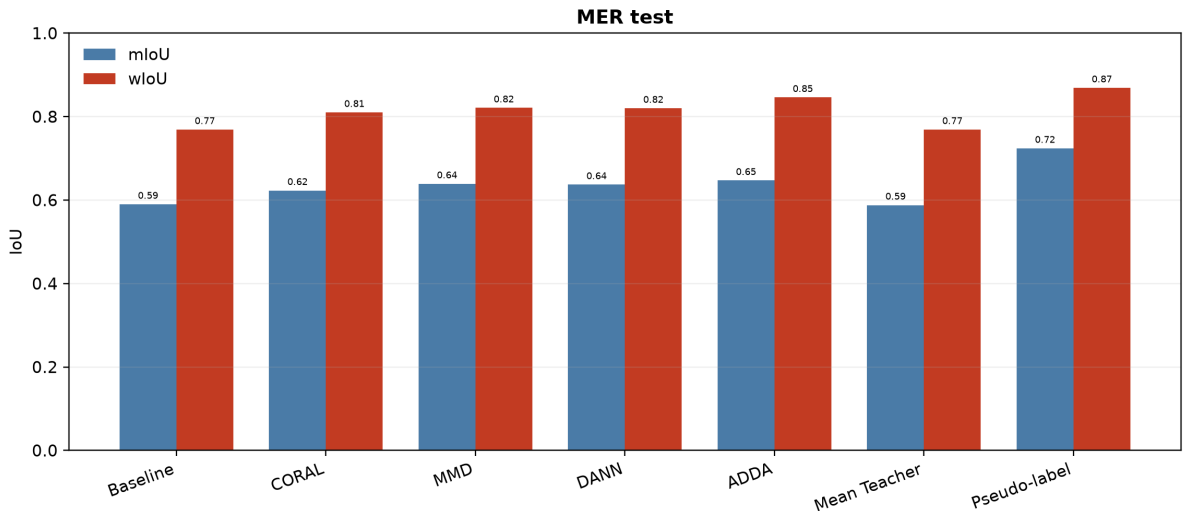


Figure 5: MER test mIoU and wIoU.

Pseudo-labeling and Mean Teacher

Pseudo-labeling improves baseline wIoU from 0.769 to **0.869** by adding confident hard labels on `mer_uda`. **Mean Teacher** instead stays at 0.769. Pseudo-labeling obtains per-class IoU 0.871 (soil), 0.847 (bedrock), 0.904 (sand) and **0.276** (*big_rock*); full scores are in Table 6.

Frequent classes exceed about 0.74 IoU in all useful runs. Pseudo-labeling gives the clearest gain on *big_rock*, from 0.036 baseline to **0.276**.

Per-class analysis

Table 6 compares baseline pretraining, Mean Teacher and pseudo-labeling on MER test. Table 5 breaks down CORAL, MMD, DANN and ADDA on MER test with per-class IoU, mIoU and wIoU. UDA methods improve the baseline on all classes but the real improvement on the rare *big_rock* class is given only by introducing a small **labelled** dataset as we can see in the pseudo-labeling results.

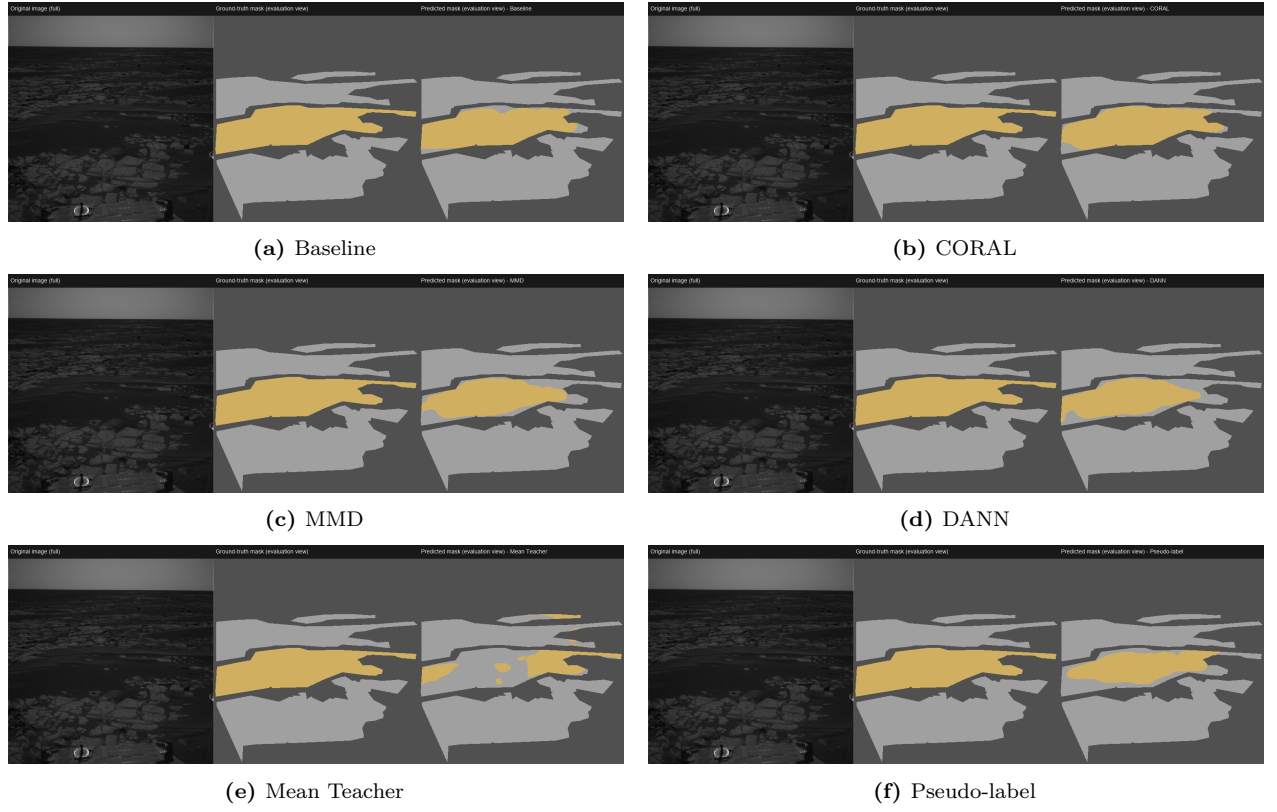


Figure 6: Qualitative MER test predictions on the same image (original / GT / prediction).

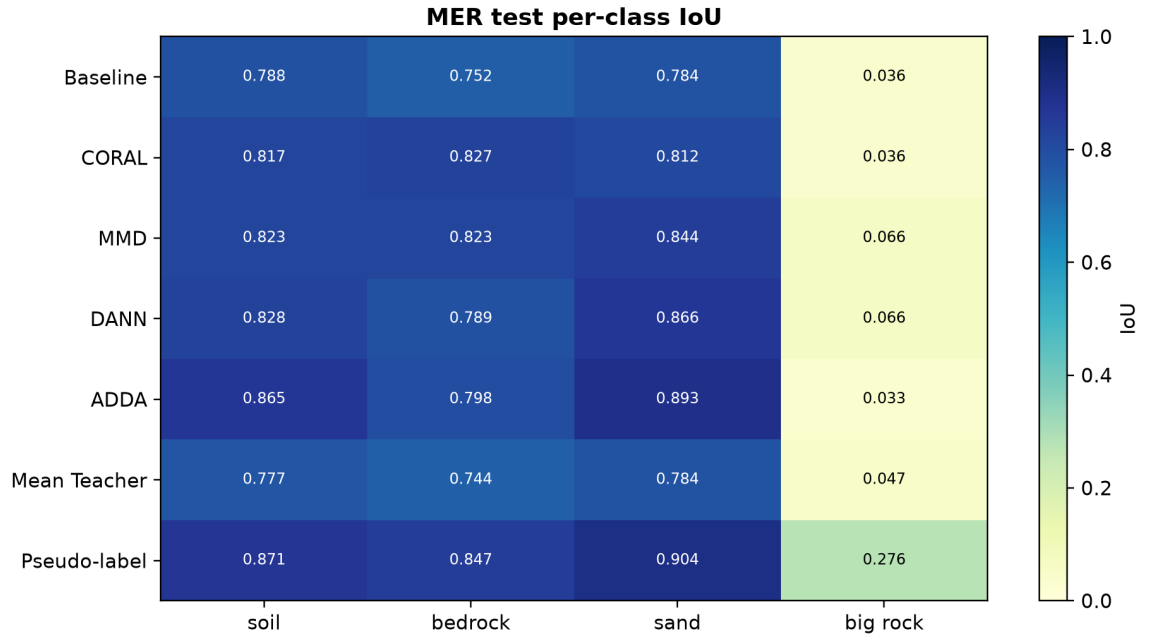


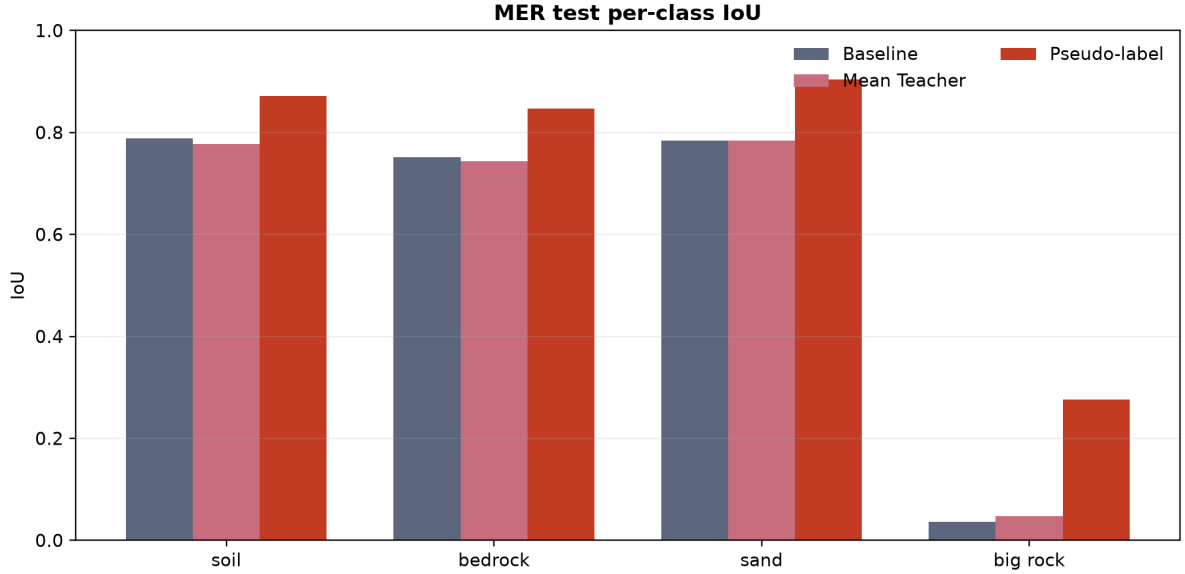
Figure 7: MER test per-class IoU.

Table 5: UDA methods on MER test.

Method	soil	bedrock	sand	big rock	mIoU	wIoU
CORAL	0.817	0.827	0.812	0.036	0.623	0.810
MMD	0.823	0.823	0.844	0.066	0.639	0.821
DANN	0.828	0.789	0.866	0.066	0.637	0.820
ADDA	0.865	0.798	0.893	0.033	0.647	0.846

Table 6: Semi-supervised methods on MER test.

Method	soil	bedrock	sand	big rock	mIoU	wIoU
Baseline	0.788	0.752	0.784	0.036	0.590	0.769
Mean Teacher	0.777	0.744	0.784	0.047	0.588	0.769
Pseudo-label	0.871	0.847	0.904	0.276	0.724	0.869

**Figure 8:** MER test per-class IoU.

Discussion of results

Unsupervised Domain Adaptation methods improve the baseline on all classes while testing in the target domain, however the problem derived from the rare *big rock* class remains difficult to solve. **Mean Teacher** matches the baseline wIoU (0.769): its consistency loss encourages agreement under perturbations but does not provide the same class-specific signal. Regarding domain adaptation, **ADDA** and **MMD** perform better than **CORAL** and **DANN**.

Conclusions

We addressed cross-rover Mars NavCam segmentation with a ResNet34-U-Net pretrained on MSL and adapted to a target domain using feature-level UDA, and verified how introducing a small labelled subset of the target domain can further improve the performance on the AI4MARS dataset.

Main findings

1. UDA raises MER test wIoU to 0.821 (MMD), 0.820 (DANN) and 0.810 (CORAL). **ADDA** yields the highest UDA wIoU (**0.846**).
2. **Pseudo-labeling** gives the best result: **0.869 MER test wIoU**, using **200 labeled** and **2000 unlabeled** MER images.
3. **Mean Teacher** uses the same 200 labels but reaches 0.769 wIoU; soft consistency underperforms hard pseudo-label cross-entropy in this setting.
4. *big_rock* remains difficult (about 1% pixels) without labels, but pseudo-labeling raises its IoU from 0.036 to **0.276**.

Future work

1. We aligned only **enc4**. Output-space or multi-scale UDA (e.g. AdaptSegNet [11]) and alignment of shallow skip features may reduce the MSL/MER appearance gap left in early encoder stages.
2. SAM or CLIP-style encoders could supply pseudo-masks or semantic targets on unlabeled MER frames without replacing the lightweight U-Net deploy path.
3. **m2020_val** (Perseverance) is held out in our protocol; evaluating whether MSL→MER models transfer to M2020 without retraining would test cross-mission robustness beyond the Spirit/Opportunity target.
4. Results rely on a single split seed. Repeating adaptation with multiple seeds and reporting mean $\pm$ std would quantify variance, especially for noisy *big_rock* scores.

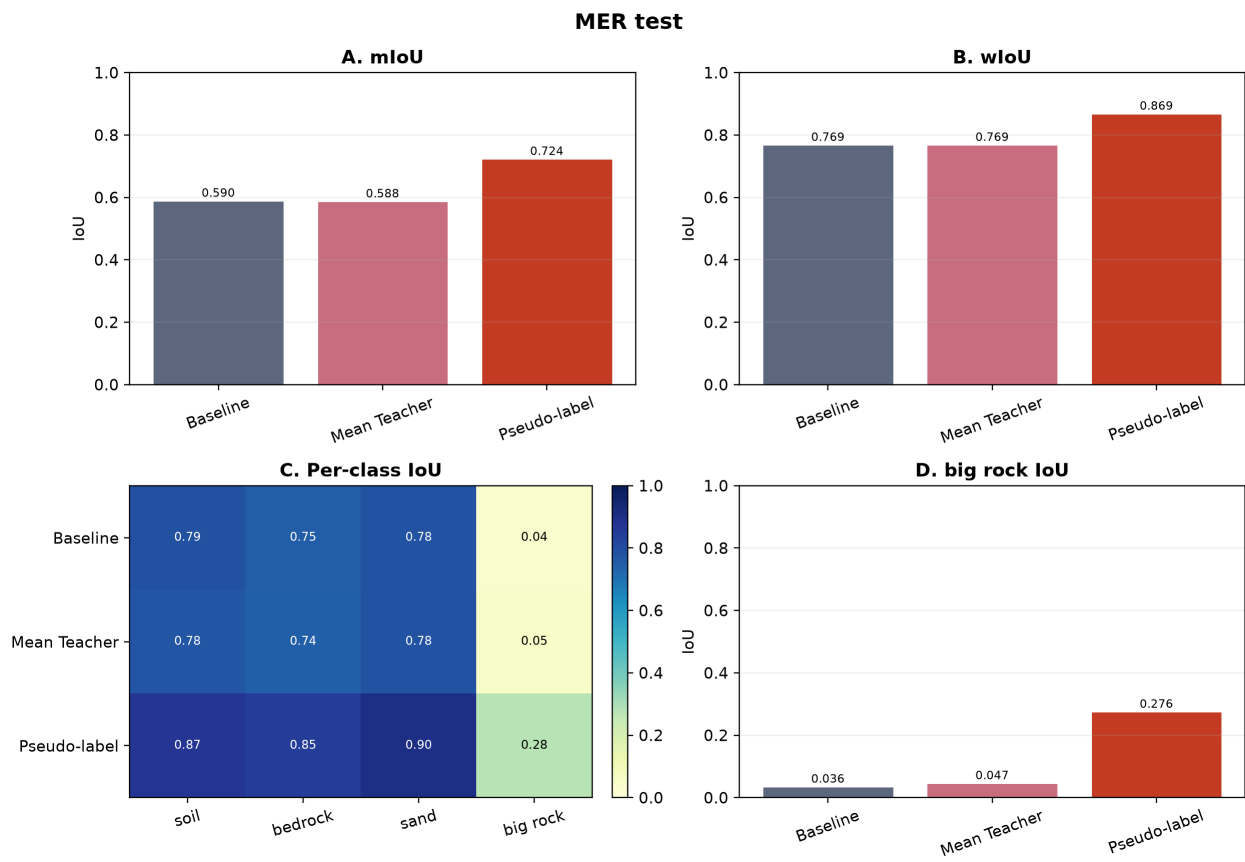


Figure 9: MER test summary.

References

- [1] A. Azari, B. Chen, A. Daftry, et al. AI4MARS: A dataset for terrain-aware autonomous driving on Mars. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [2] Y. Ganin, E. Ustinova, H. Ajakan, et al. Domain-adversarial training of neural networks. *JMLR*, 17(1):2096-2030, 2016.
- [3] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf and A. Smola. A kernel two-sample test. *JMLR*, 13:723-773, 2012.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [5] K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [6] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- [7] O. Ronneberger, P. Fischer and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.
- [8] B. Sun and K. Saenko. Deep CORAL: Correlation alignment for deep domain adaptation. In *ECCV Workshops*, 2016.
- [9] E. Tzeng, J. Hoffman, K. Saenko and T. Darrell. Adversarial discriminative domain adaptation. In *CVPR*, 2017.
- [10] A. Tarvainen and H. Valpola. Mean teachers are better role models. In *NeurIPS*, 2017.
- [11] Y.-H. Tsai, W.-C. Hung, S. Schultter, M. Yang, M.-H. Yang and M. Chandraker. Learning to adapt structured output space for semantic segmentation. In *CVPR*, 2018.
- [12] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollar and R. Girshick. Segment Anything. In *ICCV*, 2023.
- [13] D.-H. Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, 2013.
- [14] funktion. DANN: PyTorch implementation of Domain-Adversarial Training of Neural Networks. <https://github.com/funktion/DANN>, 2019.
- [15] ayushtues. ADDA_pytorch: PyTorch implementation of Adversarial Discriminative Domain Adaptation. https://github.com/ayushtues/ADDA_pytorch, 2019.
- [16] iBelieveCJM. pseudo_label-pytorch: PyTorch implementation of Lee pseudo-labeling. https://github.com/iBelieveCJM/pseudo_label-pytorch, 2018.
- [17] Curious AI. Mean Teacher: PyTorch/TensorFlow implementation of Tarvainen & Valpola. <https://github.com/CuriousAI/mean-teacher>, 2017.